

1 colibri2: A CP solver for SMT problems

2 François Bobot ✉ 

3 Université Paris-Saclay, CEA, List, F-91120, Palaiseau, France

4 Hichem Rami Ait-El-Hara ✉ 

5 OCamlPro SAS, 21 Rue de Chatillon, 75014, Paris, France

6 Bruno Marre ✉

7 Université Paris-Saclay, CEA, List, F-91120, Palaiseau, France

8 — Abstract —

9 Constraint Programming (CP) solvers offer strong reasoning capabilities based on domains, yet
10 they are rarely used as drop-in replacements for SMT solvers in program verification. Although
11 these approaches may appear at odds, in practice the SMT ecosystem is structured around the
12 SMT-LIB language and the SMT-COMP competition rather than the SAT Modulo Theories (SMT)
13 architecture itself.

14 This paper investigates how CP solvers centered on domain-based reasoning can effectively address
15 SMT problems. We present **colibri2**, a complete reimplement of the CP solver **COLIBRI**,
16 which previously won several floating-point arithmetic tracks of the SMT-COMP. **colibri2** is
17 designed to be competitive with state-of-the-art SMT solvers, and to address challenges typical of
18 SMT benchmarks, including infinite and dense domains, slow convergence, and large theories. We
19 show how to use domains and filtering to implement reasoning on usual SMT theories, including
20 algebraic data types, congruence closure, arithmetic, and quantifiers. Experiments on the SMT-LIB
21 quantifier-free floating-point benchmarks show that **colibri2** is competitive with **COLIBRI** and the
22 SMT solvers **cvc5** and **bitwuzla**.

23 **2012 ACM Subject Classification** Computing methodologies → Theorem proving algorithms

24 **Keywords and phrases** Constraint Programming, CP solver, SMT solver, scheduling, propagation,
25 domains

26 1 Introduction

27 Constraint Programming (CP) solvers have been used for program verification for more than
28 twenty years [24, 31]. While they are often implemented as libraries within verification tools,
29 they can also serve as external solvers through modeling languages such as MiniZinc [45].
30 However, in program verification the SMT-LIB format [13] and its associated theories [10]
31 have become the de facto standard interface for interacting with automated reasoning tools [6].
32 This is particularly true in deductive verification [29], where models of programming languages
33 and data structures rely heavily on axiomatizations and quantified formulas, features that
34 are typically not supported directly in CP solvers. Moreover, as different solvers exhibit
35 complementary strengths and weaknesses, relying on a standard interface makes portfolio
36 approaches particularly successful [17].

37 Historically, however, solvers supporting the SMT-LIB format have mainly been SAT
38 Modulo Theories (SMT) solvers [47]. While SMT solvers provide strong capabilities, they also
39 exhibit a different trade-off. In practice two weaknesses are particularly evident. First, the
40 transfer of information between theories (e.g., between machine arithmetic and mathematical
41 integers, or between floating-point and real arithmetic) can be hindered by the theory-
42 combination frameworks [44, 53] on which SMT solvers rely. Second, the design of SMT
43 solvers prioritizes completeness, sometimes at the expense of performance on common cases.

44 These limitations suggest that alternative reasoning strategies may offer practical ad-
45 vantages. Abstract interpretation [42] is a prominent example: by propagating bounds and

approximating program behaviors, it can quickly establish properties, such as numeric bounds or pointer validity, that SMT solvers may struggle to prove even in simple cases. For instance, the property shown in Listing 1 can be established immediately by bound propagation. The same holds for $i \in \{4, 6, 8\}$, whereas the SMT solvers `bitwuzla`[46] (version 0.8.2) and `cvc5` [7] (version 1.3.2) exhibit a linear and exponential behavior, respectively, with `cvc5` requiring 225s for $i = 8$. Recent work used domains in SMT solvers [4].

In this paper, we argue that CP solvers centered on domain-based reasoning are well suited to solving SMT problems. Furthermore, by supporting the SMT-LIB format and its theories, CP solvers can integrate into the SMT ecosystem and benefit from its extensive benchmarks and applications.

We support this claim with `colibri2`, a new CP solver designed for SMT problems. `colibri2` is a reimplement of the CP library and solver `COLIBRI`, which was originally developed as part of a `SCADE` program verification tool. After adding support for parsing SMT-LIB files, `COLIBRI` achieved several wins in floating-point logics in recent editions of the SMT Competition (SMT-COMP) [8]. Its development was guided both by feedback from users of the standalone version and by the challenge of solving the yearly SMT-LIB benchmarks submitted to the competition. A full reimplement was nevertheless required. The original implementation relied on the `ECLiPSe Prolog` framework [51], whose scheduling mechanisms do not provide the fairness and flexibility needed for our purposes. As a result, the implementation relied on ad hoc workarounds that hindered maintenance and further development. In addition, `COLIBRI` lacks support for axiomatization.

`colibri2` is implemented in `OCaml`, whose expressive type and module system [39] enables clear modular boundaries between propagators and improves maintainability. The framework has been redesigned from the ground up with architectural choices tailored to solving SMT problems using CP techniques. In particular, `colibri2` supports theories that are rarely available in CP solvers, including quantifiers [26] and algebraic data types (ADTs) [11], both integrated through domain-based reasoning. We also implement existing decision procedures using domains, such as the decision procedure for Shostak theories [53].

The remainder of the paper is organized as follows. Sections 2 and 3 provide context on the SMT-LIB ecosystem and the notations used throughout the paper. Section 4 presents the high-level architecture of `colibri2` and formalizes its union-find environment and scheduling phases. Section 5 describes the implementation of standard SMT theories—including Shostak theories, Algebraic Data Types, and Quantifiers—within a domain-based framework. Section 6 details the specific propagators and acceleration techniques used for floating-point arithmetic. Finally, Section 7 evaluates `colibri2` against state-of-the-art solvers on both quantifier-free and quantified floating-point benchmarks, as well as on proof obligations arising from formal program verification.

2 SMT-LIB Community

SMT usually refer to DPLL(T) [30], a reasoning framework where a SAT solver is linked to decision procedures. However, the capabilities of SMT solvers, are heavily influenced by the SMT community they live in. The SMT community really designates a language SMT-LIB [13], predefined theories [10] and a competition SMT-COMP [9].

Contrary to the MiniZinc challenges, which separate the benchmarks by the individual constraints used, the SMT-COMP separate benchmarks by the logics they use (linear arithmetic, non-linear arithmetic, floating-point). Each of them defines many symbols, more than 30 in the case of floating-point logic (FP).

■ **Listing 1** A toy example of an SMT-LIB file. Using only floating point operations and given the sequences $x_i = x_{i-1} \times y_{i-1}$, $y_i = \frac{x_{i-1} + y_{i-1}}{2}$, for any $i \geq 0$, $|x_0|, |y_1| \leq 1 \implies |x_i|, |y_i| \leq 1$. The following file is the negation of the case $i = 2$. The solver will look for values for x_0 and y_0 that satisfy the assertions and the definitions (a counter-example). The unsatisfiability proves the case $i = 2$.

```
(set-logic QF_FPLRA)
(define-fun one () Float64 ((_ to_fp 11 53) RNE 1.0))
(define-fun two () Float64 ((_ to_fp 11 53) RNE 2.0))

(declare-fun x0 () Float64)
(declare-const y0 Float64) ; alias for declare-fun ()

(assert (fp.leq (fp.abs x0) one))
(assert (fp.leq (fp.abs y0) one))

(define-fun x1 () Float64 (fp.mul RNE x0 y0))
(define-fun y1 () Float64 (fp.div RNE (fp.add RNE x0 y0) two))

(define-fun x2 () Float64 (fp.mul RNE x1 y1))
(define-fun y2 () Float64 (fp.div RNE (fp.add RNE x1 y1) two))

(assert (not(and (fp.leq (fp.abs x2) one) (fp.leq (fp.abs y2) one))))

(check-sat)
```

More importantly, the constants are not only the constant of the built-in theories (e.g. 1, true), but also terms that can potentially take all the possible value. They are often called variables in CP, expression which is used for quantified variable in SMT-LIB. However differently from MiniZinc [54] all the functions are total. Mathematical partial functions like division $\frac{x}{y}$ are made total by not constraining the value of the models when $y = 0$. So for any real c , $\frac{1}{0} = c$ is satisfiable, so is $\frac{1}{0} \neq \frac{2}{0}$. Still $\frac{1}{0} \neq \frac{1}{0}$ is unsatisfiable.

Uninterpreted sorts and functions are symbols that lack a predefined interpretation in any specific theory. Consequently, the cardinality of an uninterpreted sort is not fixed and may be infinite. While these symbols have no inherent semantics beyond the axiom of congruence, they can be further constrained by the user through assertions or quantified formulas.

3 Notations

In this paper we are using the following notations. c are constants, v are binder variables (used for $\forall, \exists$ binders), F is an input term, $\mathcal{G}$ is the set of ground terms, the set of function symbols is Symb . We follow the notation and definitions for the semantics of first-order logic and theory-specific models as presented in [12]. This formulation is related to the semantics of constraints in Constraint Logic Programming (CLP) [33]. The theory $\mathcal{T}$ we are interested in contains all the theory supported by colibri2. A model $\mathcal{A}$ is a pair of a non-empty Sets A the universe and a mapping which associates to every symbol $f \in \text{Symb}$ of arity n a mathematical function $f^{\mathcal{A}}$ from A^n to A . It allows defining an interpretation from terms to A noted $(.)^{\mathcal{A}}$.

The extensibility of colibri2 is handled using extensible functions and extensible types. In the case of extensible functions, one can register one's own function to be called when the extensible function is called. If the function returns a unit type, all registered functions

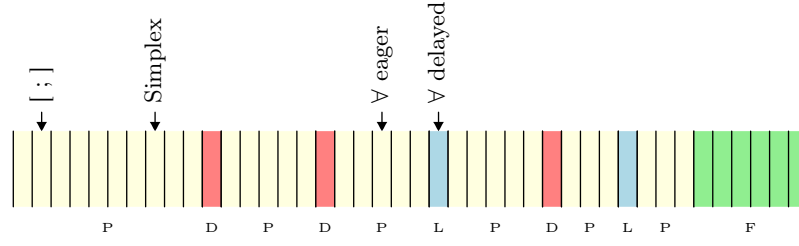
are called; otherwise, the result of the first function to return a value is used. Extensible functions can be executed immediately or deferred to be executed later, having arbitrary code executed in the middle of its own execution is harder to make right. Extensible types can be seen like an object class that code can inherit from and extend.

4 Overall Architecture

The architecture of `colibri2` is composed of various inter-dependent components, including nodes, the union-find environment, daemons, and the scheduler. `colibri2` is based on a basic CP solving architecture [48]. In particular Nodes (`Node`) are an extensible type that represent different kind of terms (and they form the equivalence classes).

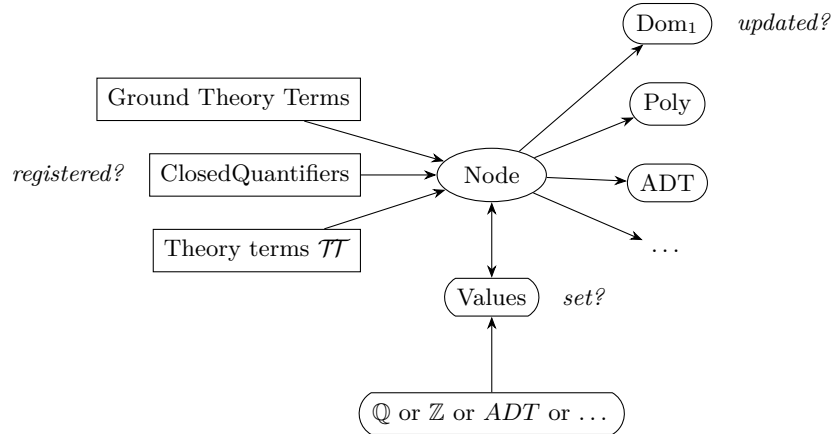
Let's follow a usual execution of `colibri2` in order to present the different part and features:

- The set of theories being used is defined at the initialization of the environment. At that time the theories register into extensible function and extensible types as needed. In fact a theory is nothing more than that in `colibri2`.
- The SMT file is parsed and typed by the OCaml library `dolmen` [22] (also used by `COLIBRI`) resulting in a list of statements (assertions, function definitions, declaration). They contain function symbol applications (e.g. logical operator, `ite`, `+`, uninterpreted function), binders ($\forall$, $\exists$, `let`).
- The statements are handled in order. Function definitions are given to one specific extensible function.
- For a given assertion, its boolean ground term is converted into a specific kind of node called *ground theory term*; recursively, every ground sub-term is similarly converted into a node. Ground term that are quantified formulas are converted into *closed quantified formula theory term* nodes.
- The node corresponding to the assertion is set to $\top$ (using an extensible function), and then *registered*: *Registering* a node schedules the *register* extensible unit function for theory terms. In the case of a ground theory term, the theory that knows about this function symbol usually registers the arguments, initializes the domain if needed, registers propagators on the domain of its node or its arguments, or schedules a *canvassing* function. Unregistered nodes can have domains or a value, but cannot be merged with another equivalence class.
- The first three phases of the scheduler, in order of priority, the propagation phase, decision phase and last-effort phase, are associated with their own queues. One phase is run only when the queue of the higher priority ones are empty.
- The scheduler is started at the propagation phase, where propagators are run, they modify domains, which schedule new propagators or others deferred functions.
- During the decision phase, the scheduler selects one of the deferred *canvassing* functions that returns a list of choices that will create as many search branches. The *canvassing* function can also indicate that it doesn't need any choice, so another function is selected. Once a decision is made the scheduler goes back in the propagation phase.
- The queue of the last-effort phase consists of propagators, but usually these propagators create new terms which would lead the propagation phase to not terminate if they were done in it.
- In a branch, once no more propagators (for the propagation and last-effort phase) or canvassing functions are deferred we go in the *fix-model* phase irremediably for this branch. The *fix-model* phase enumerates the remaining possible models. For every registered node



■ **Figure 1** Representation of the different phases of the scheduling along one search branch: propagations (P), decisions (D), last-effort (L), and fix-model (F). Some examples of propagations are positioned, interval propagation $[;]$ are deferred less time than the Simplex. Eager instantiation of quantifier are in the propagation phase when delayed instantiation are in the last-effort phase.

161 an extensible function can return the possible values according to the domains or more
 162 global information. The search of a model is done using iterative deepening, by fixing
 163 one node at a time. Propagators can still be run during this phase.
 164 ■ Once a complete model is obtained it is tested against on the ground terms using a
 165 simple extensible function of interpretation. The soundness of the sat answer depends
 166 only on this interpretation. The interpretation function could answer unknown when the
 167 interpretation is not possible or too costly (e.g. true universal quantifiers).



■ **Figure 2** Relations between different concepts at the core of colibri2 [1].

168 4.1 Nodes and the union-find environment

169 4.1.1 Nodes and theory terms

170 The architecture of colibri2 is that of a classic CP solver [48]: a union-find keeps track of
 171 the equivalence classes and their associated domains, a scheduler organizes the propagation
 172 and the search, finally, different theories define daemons that filter the domains or request
 173 the merge of equivalence classes.

174 In SMT solvers, the union-find is often extended to handle the congruence closure of the
 175 function symbols. colibri2 implements congruence closure using a domain and a daemon
 176 (cf. Section 5.5).

177 The elements of the equivalence classes are nodes noted as `Node`. They are similar to
 178 variables in CLP but are created differently.

179 As illustrated on the left of Figure 2, nodes are created from theory terms $\mathcal{T}$. In the
 180 core of the solver they are in fact exactly theory terms but they are separated from nodes
 181 outside of the core of the solver in order to keep the representative node abstract. Theory
 182 terms are defined as a recursive extensible type, which contains at least:

- ground theory term, i.e. function applications:

$$\{f(n_1, \dots, n_i) \mid f \in \text{Symb}, f \text{ is of arity } i, \forall j. n_j \in \text{Node}\}$$

- 183 ■ closed quantified formulas, $\forall \vec{v}, F(\vec{v})\sigma$.

184 Those two are used for converting recursively the input formulas coming from the parser
 185 to nodes. It is similar to purification [44].

186 The interpretation of a model $\mathcal{A}$ is extended to nodes and theory terms recursively. When
 187 the theory term type is extended, this interpretation is also extended.

188 4.1.2 State

189 At a high-level an environment `env` contains:

- 190 ■ `reprs : Node → Node`
 191 A partial mapping representing the equivalence classes of nodes, it associates each node
 192 n to the representative of its equivalence class r
- 193 ■ `doms : Node →  $\mathcal{D}$  →  $\mathcal{R}_{\mathcal{D}}$`
 194 A partial mapping of each representative node r to a mapping of each domain identifier
 195 $\mathcal{D}$ to the domain's representation for the node r if it exists.
- 196 ■ `contradiction : Bool`
 197 A flag indicates if a contradiction has been found (the implementation raises an exception
 198 in this case)

199 When a node n is registered the mapping `reprs`(n) = n is added. Naturally, a model $\mathcal{A}$ is
 200 said to satisfy `reprs` if $\forall n \in \text{Node}. (\text{env.reprs}[n])^{\mathcal{A}} = (n)^{\mathcal{A}}$.

201 4.1.3 Domains

202 Figure 2 also shows on the left that each node is associated with a set of domains. Each
 203 node kind can be associated with the different kind of domains. For example, on one hand,
 204 arithmetic nodes are associated with a domain of interval unions, which store the possible
 205 values that the node can have, as well as a domain that holds their normalized linear form
 206 representation Section 5.3. On the other hand, ADT nodes, are not associated with these
 207 two domains, but instead have a domain that represents the set of possible constructors the
 208 ADT can have Section 5.5.

209 ► **Definition 1** (Domains in colibri2). A domain $\mathcal{D}$ in colibri2 can be formalized as a
 210 tuple $\langle \mathcal{R}, P_{\mathcal{A}}, \text{equal} \rangle$ where:

- 211 ■ $\mathcal{R}_{\mathcal{D}}$: the set of domain representants.
- 212 ■ $P_{\mathcal{D}}^{\mathcal{A}} : \mathcal{R} \rightarrow 2^A$
 213 Given a model $\mathcal{A}$ and an element of the domain $d \in \mathcal{R}$, $P_{\mathcal{A}}(d)$ denotes the set of values
 214 in the universe A represented by d under $\mathcal{A}$.

215 In Definition 1, $P_{\mathcal{A}}$ depends on the model $\mathcal{A}$ in order to support relational domains,
 216 which are domains whose elements can contain other nodes. An example of such a domain is

the linear form domain, where the linear form representation r of a node can contain other nodes, therefore, to compute $P_{\mathcal{A}}(r)$, a model $\mathcal{A}$ is needed to retrieve the values of the nodes that may appear in r .

Naturally, a model satisfies env.doms , $(\text{env})^{\mathcal{A}}$, if the interpretation of a node is in the interpretation of its domain, $\forall \mathcal{D}, \forall n \in \text{Node}. (n)^{\mathcal{A}} \in P_{\mathcal{D}}^{\mathcal{A}}(\text{env.doms}(n, \mathcal{D})^{\mathcal{A}})$

The implementation of a domain need to register a function $\text{merge}_{\mathcal{D}} : \text{env} \rightarrow (n_1 : \text{Node}) \rightarrow (n_2 : \text{Node}) \rightarrow \text{env}$ which should ensures that the nodes n_1 and n_2 have the same domain $\mathcal{D}$ in env . The function must follow the following soundness property:

$$\forall \mathcal{A}, \text{env}, n_1, n_2. (\text{env})^{\mathcal{A}} \implies (n_1)^{\mathcal{A}} = (n_2)^{\mathcal{A}} \implies (\text{merge}_{\mathcal{D}}(\text{env}, n_1, n_2))^{\mathcal{A}}$$

A helper function allows to defined domains using a classical intersection function $\text{inter}_{\mathcal{D}} : \mathcal{R}_{\mathcal{D}} \rightarrow \mathcal{R}_{\mathcal{D}} \rightarrow \mathcal{R}_{\mathcal{D}}^{\perp}$. The generic version is useful in the case of relationnal domain.

The domains' merge functions are used when merging two nodes. When the merge of two nodes is requested, before updating their representatives in env.reprs , their domains are first conciliated using the domains' respective merge functions, ensuring that they have the same domain representations for all domains. Only then the nodes are merged in reprs . In **ECLiPSe Prolog** the unification occurred before the invocation of the unification handler, which removes the ability to differentiate the variables, which complicate the implementation of relationable domains.

4.1.4 Values

Variable labelling is fundamental of CP [49] for judging the satisfiability of a problem. It avoids the need of a combination framework, at the possible cost of non-termination in the case of infinite sorts.

We represent the universe $\mathcal{V}$ via a concrete extensible type Val . A node n is associated with a value $v \in \text{Val}$ through two mechanisms:

- a flat domain $\mathcal{D}_v$, where $\text{DomRepr}_{\mathcal{D}_v} = \text{Val}$, ensuring that an equivalence class contains at most one constant;
- canonical theory terms, where each v is mapped to a unique node $\bar{v}$, ensuring all nodes sharing a value are merged.

An extensible function $\text{iterate} : \text{env} \rightarrow \text{Node} \rightarrow (\text{Val})_i$ yields a sequence of potential values consistent with a node's current domains. During the fix-model phase, the scheduler explores these sequences to find a complete assignment. Satisfiability is then verified by interpreting the ground terms against this explicit model, ensuring soundness even with incomplete theory propagators.

4.2 Propagators, Daemons and Events

colibri2 uses the classic combination of events, propagators and daemons [49](§14.2.4).

Figure 2 shows the most used kinds of events:

- A new theory term is registered: used to initialize the domains of the node produced from the term and to add useful daemons associated with it.
- The domain of a node is updated: used for filtering; to propagate the changes to other nodes that depend on it, or whose domains depend on it.
- A value of some kind is registered: mainly used to set the domains of its corresponding node to a singleton.

These events can be found in two flavors: they can either wait for such an event to occur on any node, or wait for it to occur on a specific node.

Contrary to `ECLiPSe Prolog` used by `COLIBRI`, in order to help reason and implement a propagator, when it requests the merge of two nodes, the merge is not done immediately but deferred for after its execution. Similarly, as in `ECLiPSe Prolog`, when an event is triggered (e.g. change of a domain) the associated propagators are not run immediately. Only the daemon is executed immediately in a read-only environment (ensured by typing) to decide if the propagator should be deferred, removed from the event or ignored.

So a pair propagator/daemon can either be repetitive, meaning that they indefinitely wait on their event and run whenever it occurs, as many times as it occurs, or they run only once when the event occurs for the first time. They can also be parametrized to run directly after their creation if the event on which they are waiting occurred in the past, otherwise, they wait for it to occur again. They can also remember that they are already deferred for avoiding being deferred twice.

4.3 Propagation engine: Scheduler

The scheduler is a basic propagation engine and search procedure (§ 14.1 [49]). There is no idempotency detection and daemons are responsible for avoiding multi-scheduling. A great care is put on the order in which propagations are run.

Canvassing function can be registered by theories (e.g. decision on a boolean literal, which constructor is an ADT). The decisions that are needed for labelling all the nodes are handled in the specific Fix-model phase.

Unconventionally some propagations are run only when there is no more deferred *canvassing* function (e.g. quantifier instantiation which creates terms) in the Last-effort phase. Indeed quantifier instantiation propagators can lead to starvation by creating too many new terms if run during propagation. This phenomenon is also called a matching loop [38].

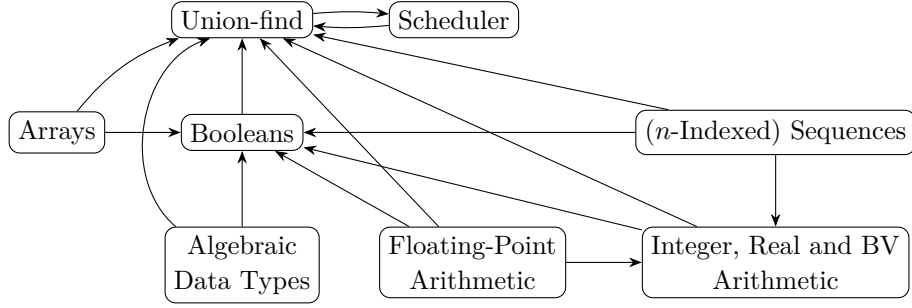
The scheduling of the propagators are ordered using an efficient time wheel data structure (similar to §6.2 in [55]). It can be seen as an infinite array of queues. Elements can be added only in the queues at index $i > 0$. The elements are dequeued only from the queue at index 0. When it is empty, all the other queues are moved one index back. Cheap deferred propagators, for example, are enqueued in index 1 (e.g. interval propagation), while costlier ones at index 64 (e.g. simplex algorithm). The amortized complexity is $O(\log(i))$ when adding an element and $O(1)$ for getting the next elements, and the gain in fairness is important.

► **Remark 2 (Slow convergence).** Fairness is important to mitigate slow convergence [19]. For example on two integers ($x \in [0; +\infty[$, $y \in [0; +\infty[$), on the constraints $x < y \wedge y < x$, a domain of interval will remove one integer at a time on the bounds of x and y : $y \in [1; +\infty[$, $x \in [2; +\infty[$,... A run of the simplex will find directly the contradiction.

Slow convergences are also mitigated using acceleration techniques [49](§16.3.5) which avoid triggering update event. For example a domain can customize that an event update is not triggered when there were too many updates after the last decision, or when the update is too small.

The fix-model phase, iterates on the values returned by the *iterate* function for each equivalence class that doesn't have any value:

■ It is easier to compute the result of a function from its arguments, than the inverse. So the iteration try to start from the leafs of the ground terms. However since a term can be equal to one of its subterm we use a weak topological order [20] to order the equivalence class.



■ **Figure 3** The interactions between the theories, the scheduler, the daemons, and the union-find in `colibri2` [1].

- 301 ■ Once all the nodes have a value, and no propagators remain to run, the model is checked
- 302 by interpretation of the ground theory terms. The soundness of the SAT answers only
- 303 depend on that. So it allows to start with incomplete propagators and reasoning for a
- 304 symbol of a theory without compromising soundness.
- 305 ■ Fairness is preserved during value iteration by using iterative deepening. First we use
- 306 only 2 values by equivalence class, then 4 if no model is found, and so on.
- 307 ■ After a certain number of iterations, by default 1000, if none of the generated models
- 308 are satisfying, the scheduler closes the branch and acts as if a conflict was detected and
- 309 backtracks to make a different decision. If, after exploring all branches, no satisfying
- 310 model is found, then `colibri2` answers with “unknown-branch-cut”, which means that
- 311 no conflict was detected, but the solver did not manage to produce a model. This result
- 312 usually indicates that some propagations are not powerful enough.

313 4.4 No Good and clause generations

314 COLIBRI do not use learning (no good or clause learning), in the first version of its reim-
 315 plementation we tried to incorporate learning in the form of an extension of MCSAT [27].
 316 However, adding learning complicated the reimplementation, so `colibri2` does not have
 317 much of it. The propagators at the last-effort phase can be reclassified to belong to the
 318 propagation phase if they lead to a conflict (before any other propagators at last-effort is
 319 run). The reason for this reclassification is that if a last-effort propagation leads to a conflict,
 320 then it would be better to perform it earlier, during propagation, in hopes of detecting the
 321 conflict sooner.

322 4.5 Organization of the theories

323 Compared to SMT, CP makes very natural to combine theories since a model is always
 324 generated. Moreover the interactions between theories are easier. Theories have access to
 325 the domains of other theories and can register daemons on terms from other theories as well.

326 Another notable difference with most SMT solvers is in boolean reasoning. In SMT
 327 solvers, it is often a special kind of reasoning, as it is done through the SAT solver and not
 328 seen as a theory. In `colibri2`, booleans are handled by a theory of booleans, which is like
 329 any other theory. The main advantage is to ensure that anything this theory can do another
 330 will have the same capability.

331 Figure 3 illustrates the interactions between the theories in `colibri2`. It shows that the
 332 boolean theory does not use any other theory, it only depends on the union-find module

which handles equivalence between terms. All the other theories depend on the theory of booleans and the union-find. They use the boolean theory to create decisions on boolean terms, notably equality and difference terms. The scheduler and union-find also interact with one another. Figure 3 also shows that the integer, real, and bit-vector arithmetic theories are joined together, since they share most of the domains they use, notably the interval domain.

Theories can store information outside of domains thanks to backtrackable data structures [5]. They are still stored in the environment, and they are backtracked at the same time than the rest of it, but they are private to the theories.

5 New implementation of usual SMT-LIB theories using domains

Using this extensible architecture, we can implement theories from the SMT-LIB mainly using domains.

5.1 Disequalities

Disequalities are handled in CP with the global constraint `Alldifferent` [50], however it is usually handled using the set of values of each distinct terms. It is not possible with many SMT theories. So we handle disequalities like in SMT [43] by a set of unique identifiers that must be disjoint when merging two equivalence class. Domains can be naturally used to store this information. The nodes of the disequalities are used as unique identifiers. We extend the algorithm to disequalities that are reified (not yet true). For example if $n_3 \equiv \text{distinct}(n_1, n_2)$, without waiting for n_3 to be true, we are adding to the domain of n_1 the mapping $n_3 \mapsto n_1$, and to n_2 the mapping $n_3 \mapsto n_2$, if n_1 and n_2 are merged, during the merge of the domain of disequalities, n_3 is found to map to different nodes, so n_3 is set to false. If the distinct was true, a contradiction is found.

A little more formally $R_{\text{Dis}} = \text{Node} \rightarrow \text{Node}$ is the set of partial maps from node to node, $P_{\text{Dis}}^A(m) = \{v \mid \forall (_, n) \in m. v = (n)^A\}$.

5.2 Uninterpreted Sort

By definition there is no particular reasoning on uninterpreted sorts, only congruence closure. During fix-model phase, the value iterated for the node of an uninterpreted sort are taken from the same set of fresh values. One value is added for each node iterated.

5.3 Shostak Theory

To summarize, a Shostak Theory as defined in [14] with signature Σ , canonizer `canon`, and solver `solve`, is able to decide if a set of equations E in Σ implies an equality $t_1 = t_2$ in E . The function `solve` returns a substitution from an equation, and `canon` normalizes a term ($\mathcal{T} \models t_1 = t_2$ iff $\text{canon } t_1 \equiv \text{canon } t_2$, where $\equiv$ is syntactic equality). The algorithm process each equation of E using `solve`, and incrementally builds an equisatisfiable sets S_i of equations where their left-hand side is a variable that appears only once. In the following, such sets of equations are considered to be substitutions.

In `colibri2`, the algorithm is implemented using a domain. The substitution is stored in a domain where R_{Sho} is the set of normalized terms, `mergeSho` uses the function `solve` to compute a new substitution and applies it on all the domains that contain those substituted variables; $P_{\text{Sho}}^A(t) = \{(t)^A\}$.

Notably, if multiple Shostak solvers are used at the same time, so each with its own domain, the combination is naturally the same as in [52], and the nodes are purification

variables. For example `colibri2` has the following linear form as domain (c and a_i are constants, n_i nodes):

- Linear arithmetic : $c + \sum a_i n_i$ (addition, multiplication)
- Bitvector : $c \oplus \sum a_i \& n_i$ (xor, logical and)
- Absolute value: $c \prod |n_i|^{a_i}$ product of absolute value of nodes, $a_i \in \mathcal{Q}$
- Sign: $\pm \prod \text{sgn}(n_i)$ product of sign of nodes

The last two are Shostak theory only on non-null nodes, otherwise solve is incomplete. We still extend the domains to possibly null nodes by keeping the unsolved equations on the side.

5.4 Union of Intervals, Congruence and Labeled Union-Find

Intervals is a common domain in CP or Abstract Interpretation. `COLIBRI` uses union of intervals that are less common because the number of intervals in a union can grow uncontrollably. In practice in `COLIBRI` it is not the case, and the gain in precision of union of intervals is significant. For example with x, y integers, and $x \neq 0$, $10 \leq y$, $z = (xy)^2$, without unions, only $y \in [10; +\infty[$ is propagated, with union $x \in]-\infty; -1] \cup [1; +\infty]$ and $z \in [10; +\infty[$ are also propagated.

Contrary to `COLIBRI` which uses floating-point numbers, `colibri2` uses algebraic numbers at the bounds of its intervals. On some examples, a few cycle of propagations created very big rational numbers or rational number with big numerator and denominator (for example convergence toward 1). The first case is handled by suppressing domain modification events when the bounds are too big. The second one has been handled by over-approximating union of interval that contains such rational number using smaller denominators.

Like `COLIBRI`, `colibri2` uses a domain of congruence [32, 37]. It only extend the remainder to algebraic number. For $q \in \mathcal{Q}$ and $a \in \mathcal{A}$ the domain represents $\{qk + a \mid k \in \mathcal{Z}\}$

In order to avoid some sources of unsoundness the implementation of union of intervals has been formally proved using Why3 [16], it is the `UnionOfInterval` benchmarks in the experimentation.

5.4.1 Algebraic number library

`colibri2` started to use `libpoly` [36] a library used by `NLSat` [35] and `cvc5` [7] which represent $\mathcal{A}$ using roots of polynomial with rational coefficient. However, some computations succumbed to high complexity: $v_0 = 0$ $v_n = \sqrt{2}(v_{n-1} + 1)$ was computed with polynomial of degree 2^n even if their minimal polynomial is of degree 2.

Another library called `Calcium` [34] based on the `flint` library, uses an additional layer on top of this representation. It represents the elements of $\mathcal{A}$ as elements of automatically extended multivariate fields. In our case simply the v_n are represented in the vector space $\mathcal{Q}\sqrt{2} + \mathcal{Q}$, where it only needs to know that $\sqrt{2}\sqrt{2} = 2$ which is determined once. So no operations on polynomials is needed. If a $\sqrt{3}$ is used in the computation the vector space becomes $\mathcal{Q}\sqrt{2} + \mathcal{Q}\sqrt{3} + \mathcal{Q}\sqrt{6} + \mathcal{Q}$. Thanks to the efficiency of `Calcium`, `colibri2` uses the algebraic numbers as its arithmetic constants instead of only rationals.

`COLIBRI` used bounds approximation for representing algebraic number, which complicates the interpretation of model and was not able to prove generally.

5.4.2 Constant difference relation

Two nodes x and y are in relation with the constant difference relation when there exists $c \in \mathcal{R}$ such that $x = y + c$. This relation has many uses, it helps notably with equality and

inequality detection, given $x = y + c$, if $c = 0$, then $x = y$ can be deduced, and if $c \neq 0$ then $x \neq y$ can be deduced. The relation can also serve for constraint propagation between arithmetic terms which have a constant difference between them.

Those relations are found using Section 5.3 and associates specifically linear forms that share the same variable part and that differ in the constant part. The constant difference constraints are represented using a labeled union-find data structure [40]. It extends union-find from one equivalence relation, to a group of relations. We use this particular union-find to store the substitution Section 5.3. It reduces the number of substitution to do since we are not substituting linear forms that share the same variable part.

As for union-find, the set of nodes in relations (here with the constant difference relation) can share information. Given a relation class, in which the representative is n_1 , and n_2 is a non-representative element of that class in relation $n_2 = n_1 + c$. We can avoid storing the interval domain of n_2 , when accessing the domain of n_2 , the domain of n_1 added by c is returned. Conversely the update of the domain of n_2 by an interval I , is done by updating the domain of n_1 by $I - c$.

In practice, sharing the union of interval domain like this one has allowed for better propagations and proving some tests that were not proven before, but it is also more prone for slow convergence and is detrimental on some more complex tests.

5.5 Algebraic data types

Algebraic data types, quantifier instantiation and congruence closure all need to keep track of the shape of the terms of an equivalence class and in which term they appear. Domains are very handy for representing this information, and the merge operation can naturally implement the usual algorithm.

The domain of algebraic data types keep track of the different constructor possible and when only one is possible it keeps track for each field of the constructor the node it projects into. Let an ADT `two_lists` that contain twice the constructors of a list:

```
(declare-datatypes ((list 0)) ((
  (nil1) (cons1 (car1 S) (cdr1 (list S)))
  (nil2) (cons2 (car2 S) (cdr2 (list S))) )))
```

For example if `((_ is cons1) n_1)` is true, the domain of ADT of n_1 is `{cons1()}` and in the other case it is `{nil1; nil2; cons2}`. If a theory term `(car1 n_1)` associated to the node n_2 appears in the first case, the domain of n_1 becomes `{cons1(car1  $\mapsto$   $n_2$ )}`. In the second case nothing is propagated, because `car1` is in this case uninterpreted.

The function merge for this domain, intersect the set of possibilities, and when both case are equal the common field projection are requested to be merged (`{cons1(car1  $\mapsto$   $n_1$ )}` and `{cons1(car1  $\mapsto$   $n_2$ )}` are merged into the first one, and the merge of n_1 and n_2 is requested)

COLIBRI does not implement algebraic data types.

5.6 Quantifier Instantiation and Congruence Closure

The first way colibri2 handles true universal quantifiers, is to generate patterns (subterms that contains all the variables) for each quantified formula. Matching all the existing ground term on all the patterns is too costly. We follow the heuristic of the paper Efficient E-Matching [26] for judging during a merge if a new matching is possible. It associates for every equivalence class t , every symbol f it associates two sets of $\mathcal{G}$ term, $parents_f(t_1)$ (the terms $f(\dots, t, \dots)$) and $apps_f(t)$ (the $f(\dots)$ that are in t). So in colibri2, we use a domain

which is a pair of mapping which corresponds to $(f \mapsto \text{parents}_f(n), g \mapsto \text{apps}_g(n))$. During merge the criteria described in [26] can be used, to start the matching.

Since `colibri2` does not have learning it is important to propagate as much information as possible. So it eagerly instantiates a quantifier when its body after substitute the new terms already exists modulo congruence closure. We are applying congruence closure at the same time than we are substituting the formula. We are not considering that builtin functions ($\implies, \vee, \wedge, \leq, +$) are creating new terms when they appear at toplevel of the substituted formula. If n_1, n_2, n_3 are existing, we want $n_1 \implies n_2 \leq n_3$ to be eagerly applied, and since those symbols are not used in patterns we are not creating matching loops.

Congruence closure is handled in a similar way that this axiom $\forall x_1, x_2. x_1 = x_2 \implies f(x_1) = f(x_2)$ would be matched using the multi-pattern $[f(x_1), f(x_2)]$. A congruence closure is possible if when two terms t_1 and t_2 are merged with $\text{parents}_f(t_1)$ and $\text{parents}_f(t_2)$ non disjoint sets. $\text{parents}_f(t_1)$ and $\text{parents}_f(t_2)$ indicates the ground terms to consider. The algorithm is very similar at the end to the congruence closure done in [25].

The second way `colibri2` handles quantifiers is used for proving that a universal quantification is false. The need arises from benchmarks crafted for the paper [21] that appear in `FPArith`. It is still a preliminary work only applied on floating points. It consists in instantiating quantifiers with fresh variable y_0 , waiting for the propagation and decision to eliminate values in the domain of y_0 , creating a new fresh variable y_1 whose domain starts with the complementary domain of y_0 , and so on. It is not complete but different quantifiers can be treated at the same time, and it has very good results in the experimentation.

`COLIBRI` does nothing for true universal quantification.

5.7 Arrays and Sequences

`colibri2` also supports the theories of arrays and a subset of the theory of sequences. In fact, two decision procedures have been implemented in `colibri2` for the theory of arrays, the first is an implementation of the combinatory array logic [28]. The second is based on the weakly equivalent arrays decision procedure [23]. In fact, this one is included within a calculus that was developed for the theory of n-indexed sequences [3, 2], which `colibri2` also supports. `colibri2`'s support for the theory of sequences is implemented by encoding sequence terms into the theory of n-indexed sequences and then using the calculus for n-indexed sequences to reason over them.

6 Reimplementation of the Floating-Point number theories of COLIBRI

The floating point reasoning of `COLIBRI` has been described in [41]. However it is interesting to see the impact of the different on the benchmark of the SMT-LIB for the logic `QF_*FP*` (`FPArith` division).

6.1 Propagation

The main domain used is union of intervals of floating point number with an additional flag indicating if NaN can be present. In `colibri2` the floating-point number are implemented using an extraction of the coq library `Flocq` [18]. Forward and backward propagation of the floating-point operations and conversions between other types (`fp.to_real, (_ to_fp 11 42)`) are implemented. They have not yet been formally proved, but most of them are tested using random inputs.

The union of interval is especially important due to the presence of predicate like `fp.isNormal` which when true creates a hole in the middle of the domain. This domain is already able to prove a large part of the benchmarks.

6.2 Floating-point and Real Comparisons

When two floating-point numbers are finite their comparison in floating points is equivalent to there comparison in real: $(\text{fp.leq } x \ y) \equiv (\text{fp.to_real } x) \leq (\text{fp.to_real } y)$. Adding this simple rule allows to prove 599 additional goals, mainly in `QF_BVFP`¹.

6.3 Linearization of the rounding function

Most of the floating-point operations are defined using a rounding function $o(\cdot)$ applied on the corresponding real operations (if $(\text{fp.add } x \ y)$ is a finite number then it is $o(x + y)$). The distance between x and $o(x)$ depends on the magnitude of x and the size of the mantissa of the floating-point considered [15]. For example if $o(x)$ is a positive normal floating point number, we have $Cx \leq o(x) \leq C'x$ for some constants C, C' . Those additional constraints can be given to a Simplex on reals. Mainly benchmarks in `QF_FP` (35) benefits from this operation when enabled in `colibri2`. The constants C and C' have often big numerators and denominators since they represent a small error, the computation inside the simplex using those rationals is responsible for 40% of the runtime in big benchmarks.

6.4 Monotony of the rounding function

The definition of floating-point operations allows also to use the monotonicity of the rounding function. For example, if M is a rounding mode, x and y are not NaN and x is a positive floating point number, $(\text{fp.leq } y \ (\text{fp.add } M \ x \ y))$ is true. Moreover if $(\text{fp.add } M \ x \ y)$ and $(\text{fp.add } M \ x \ z)$ are not NaN, then $\text{fp.leq } y \ z$ implies that $(\text{fp.leq } (\text{fp.add } M \ x \ y) \ (\text{fp.add } M \ x \ z))$

7 Benchmarks

We compared `colibri2`², `COLIBRI`³, `cvc5`(version 1.3.2), `bitwuzla` (version 0.8.2) on the SMT-LIB benchmarks of the category `QF_FPArith` and `FPArith` of the SMT-COMP and `UnionOfInterval`. The virtual prover `cvc5+bitwuzla` is the best result of them.

- `QF_FPArith` regroups all the 58123 benchmarks without quantifiers that uses floating-point numbers in the SMT-LIB 2025. It is the historical category of `COLIBRI`
- `FPArith` regroups all the 4341 benchmarks with quantifiers that uses floating-point numbers in the SMT-LIB 2025. The goal is to measure the improvements on quantifiers
- `UnionOfInterval` is the proof of the library of union of interval used in `colibri2` described in Section 5.4 using `Why3` [17] and `Why3find` [?]. It is used to compare `colibri2` on program verification and features: ADT, function recursively defined, non-linear real and integer arithmetic, quantifiers, polymorphism.

¹ `colibri2` has also some reasoning between bitvectors and real or integers which can help

² The used version of `colibri2` corresponds to the commit with the hash: `8d148fff62baf0f5783eec772769f1f29e1d7fc1`.

³ Available at the `COLIBRI` website: <https://colibri.frama-c.com/download.html>

solver	colibri2	COLIBRI	cvc5	bitwuzla	cvc5+bitwuzla
solved for QF_FPArith	57770	57785	57881	57884	58051
improved with colibri2	-	+250	+166	+172	+37
worsen with colibri2	-	-264	-275	-284	-316
improved on supported	-	+21	+67	+41	+37
solved for FPArith	1464	231	2492	2101	2940
improved with colibri2	-	+1414	+43	+792	+30
worsen with colibri2	-	-181	-1070	-1427	-1505
improved on supported	-	+660	+43	+38	+30
solved for UnionOfInterval	1660	-	1605	-	1605

Table 1 Comparison of colibri2 against COLIBRI, cvc5, and bitwuzla. *Better with colibri2* are the number of benchmarks that colibri2 solves but not the solver in the column (*worse* is the contrary). *Better on supported* is the same except the benchmark not supported by the other solver are ignored.

Experiments were conducted using six cluster nodes, each equipped with 72 cores at 3GHz and 187GB of RAM. 30 cores per node were used, with a time limit of one minute and a memory limit of 4GB per problem.

A summary of the comparison is in Table 1. Thanks to its architecture that streamlines the addition of new theories, colibri2 supports all the theories of QF_FPArith. So on the full benchmark colibri2 has similar results than the other solvers compared with around 200 benchmarks proved by colibri2 and not proved by the other, *improved*, or the inverse, *worsen*. However, if we restrict to the benchmark supported by the other solver, without ADT for bitwuzla and COLIBRI and with only classic 32bit and 64bit floating point for cvc5 and COLIBRI, colibri2 only improve on around 40 benchmarks. Still it is able to prove 37 benches that neither cvc5 nor bitwuzla can prove.

On FPArith colibri2 shows significant improvements against COLIBRI, mainly due to the refutation of universal quantification^{5.6}. Still compared to the other solver there is still a major difference.

On UnionOfInterval, colibri2 proves more proof obligations generated by Why3 than cvc5. More importantly even if we use also the solvers CVC4, Alt-Ergo and Z3, and use the default proof strategy, some functions are not proved. This shows the interest of colibri2 for program verification through high-level language such as Why3.

8 Conclusion

We presented colibri2, a reimplementation of COLIBRI. Our results show that CP techniques are effective for solving SMT-LIB problems and suggest that dedicated CP solvers should be included in solver portfolios, particularly in the context of program verification. We also showed how theories commonly used in the SMT community can be implemented naturally using domain-based reasoning. Some features and propagators from COLIBRI still need to be ported to colibri2 to achieve similar efficiency across all supported theories. We hope this work will foster new CP solvers to be part of the SMT community.

566 — References

- 567 1 Hichem Rami Ait El Hara. *Theory of sequences tailored for program verification*. Theses,
568 Université Paris-Saclay, October 2025. URL: <https://theses.hal.science/tel-05383515>, doi:
569 10.70675/d119d789z344ez4a4ez8463z1f322a4484c7.
- 570 2 Hichem Rami Ait-El-Hara, François Bobot, and Guillaume Bury. Reasoning over n-indexed
571 sequences in smt. *Acta Inf.*, 62(3), August 2025. doi:10.1007/s00236-025-00496-w.
- 572 3 Hichem Rami Ait-El-Hara, François Bobot, and Guillaume Bury. An SMT Theory for n-Indexed
573 Sequences. In Giles Regeer and Yoni Zohar, editors, *Proceedings of the 22nd International
574 Workshop on Satisfiability Modulo Theories*, volume 3725 of *CEUR Workshop Proceedings*, pages
575 64–74, Montreal, Canada, July 2024. CEUR. URL: <https://ceur-ws.org/Vol-3725/#short13>.
- 576 4 Hichem Rami Ait-El-Hara, Guillaume Bury, Basile Clément, and Pierre Villemot. Constraint
577 Propagation for Bit-Vectors in Alt-Ergo. In Jochen Hoenicke, Mikoláš Janota, Aina Niemetz,
578 and Sophie Tourret, editors, *Joint Proceedings of the 23rd International Workshop on Satis-
579 fiability Modulo Theories and the 16th Pragmatics of SAT International Workshop*, volume
580 4008 of *CEUR Workshop Proceedings*, pages 65–76, Glasgow, UK, August 2025. CEUR. ISSN:
581 1613-0073. URL: https://ceur-ws.org/Vol-4008/#SMT_paper20.
- 582 5 Clément Allain, Basile Clément, Alexandre Moine, and Gabriel Scherer. Snapshottable stores.
583 *Proc. ACM Program. Lang.*, 8(ICFP), August 2024. doi:10.1145/3674637.
- 584 6 Roberto Amadini, Mak Andrlon, Graeme Gange, Peter Schachte, Harald Søndergaard, and
585 Peter J. Stuckey. Constraint Programming for Dynamic Symbolic Execution of JavaScript.
586 In Louis-Martin Rousseau and Kostas Stergiou, editors, *Integration of Constraint Program-
587 ming, Artificial Intelligence, and Operations Research*, volume 11494, pages 1–19. Springer
588 International Publishing, Cham, 2019. doi:10.1007/978-3-030-19212-9_1.
- 589 7 Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai
590 Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex
591 Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar.
592 cvc5: A versatile and industrial-strength SMT solver. In Dana Fisman and Grigore Rosu,
593 editors, *Tools and Algorithms for the Construction and Analysis of Systems - 28th International
594 Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and
595 Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I*,
596 volume 13243 of *Lecture Notes in Computer Science*, pages 415–442, Munich, Germany, 2022.
597 Springer. doi:10.1007/978-3-030-99524-9_24.
- 598 8 Clark Barrett, Leonardo De Moura, and Aaron Stump. SMT-COMP: Satisfiability Modulo
599 Theories Competition. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg,
600 Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan,
601 Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi,
602 Gerhard Weikum, Kousha Etessami, and Sriram K. Rajamani, editors, *Computer Aided
603 Verification*, volume 3576, pages 20–23. Springer Berlin Heidelberg, Berlin, Heidelberg, 2005.
604 doi:10.1007/11513988_4.
- 605 9 Clark Barrett, Morgan Deters, Leonardo De Moura, Albert Oliveras, and Aaron Stump.
606 6 Years of SMT-COMP. *Journal of Automated Reasoning*, 50(3):243–277, March 2013.
607 doi:10.1007/s10817-012-9246-5.
- 608 10 Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The Satisfiability Modulo Theories Library
609 (SMT-LIB), 2016.
- 610 11 Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The SMT-LIB Standard: Version 2.6.
611 Technical report, Department of Computer Science, The University of Iowa, 2017. Available
612 at www.SMT-LIB.org.
- 613 12 Clark Barrett, Roberto Sebastiani, Sanjit Seshia, and Cesare Tinelli. Satisfiability modulo
614 theories. In Armin Biere, Marijn J. H. Heule, Hans van Maaren, and Toby Walsh, editors,
615 *Handbook of Satisfiability, Second Edition*, volume 336 of *Frontiers in Artificial Intelligence
616 and Applications*, chapter 33, pages 825–885. IOS Press, February 2021. URL: [http://theory.
617 stanford.edu/~barrett/pubs/BSST21.pdf](http://theory.stanford.edu/~barrett/pubs/BSST21.pdf).

- 618 13 Clark Barrett, Aaron Stump, and Cesare Tinelli. The SMT-LIB Standard: Version 2.0. In A.
619 Gupta and D. Kroening, editors, *Proceedings of the 8th International Workshop on Satisfiability*
620 *Modulo Theories (Edinburgh, UK)*, 2010.
- 621 14 Clark W. Barrett, David L. Dill, and Aaron Stump. A generalization of shostak’s method
622 for combining decision procedures. In *FroCoS ’02*, pages 132–146. Springer, 2002. doi:
623 10.5555/646821.706603.
- 624 15 Mohammed Said Belaid, Claude Michel, and Michel Rueher. Boosting local consistency
625 algorithms over floating-point numbers. In Michela Milano, editor, *Principles and Practice of*
626 *Constraint Programming*, pages 127–140, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- 627 16 François Bobot, Jean-Christophe Filliâtre, Claude Marché, and Andrei Paskevich. Why3:
628 Shepherd your herd of provers. In *Boogie 2011: First International Workshop on Intermediate*
629 *Verification Languages*, pages 53–64, 2011.
- 630 17 François Bobot, Jean-Christophe Filliâtre, Claude Marché, and Andrei Paskevich. Why3:
631 Shepherd your herd of provers. *Boogie 2011: First International Workshop on Intermediate*
632 *Verification Languages*, 05 2012.
- 633 18 Sylvie Boldo and Guillaume Melquiond. Flocq: A Unified Library for Proving Floating-point
634 Algorithms in Coq. In Elisardo Antelo, David Hough, and Paolo Ienne, editors, *Proceedings of*
635 *the 20th IEEE Symposium on Computer Arithmetic*, pages 243–252, Tübingen, Germany, July
636 2011. URL: <https://hal.inria.fr/inria-00534854>, doi:10.1109/ARITH.2011.40.
- 637 19 Lucas Bordeaux, Youssef Hamadi, and Moshe Y. Vardi. An analysis of slow convergence
638 in interval propagation. In Christian Bessière, editor, *Principles and Practice of Constraint*
639 *Programming – CP 2007*, pages 790–797, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
640 doi:10.1007/978-3-540-74970-7_56.
- 641 20 François Bourdoncle. Efficient chaotic iteration strategies with widenings. In Dines Bjørner,
642 Manfred Broy, and Igor V. Pottosin, editors, *Formal Methods in Programming and Their*
643 *Applications*, pages 128–141, Berlin, Heidelberg, 1993. Springer. doi:10.1007/BFb0039704.
- 644 21 Martin Brain, Aina Niemetz, Mathias Preiner, Andrew Reynolds, Clark Barrett, and Cesare
645 Tinelli. Invertibility Conditions for Floating-Point Formulas. In Isil Dillig and Serdar Tasiran,
646 editors, *Computer Aided Verification*, volume 11562, pages 116–136. Springer International
647 Publishing, Cham, 2019. doi:10.1007/978-3-030-25543-5_8.
- 648 22 Guillaume Bury. Dolmen: A Validator for SMT-LIB and Much More. In Alexander Nadel
649 and Aina Niemetz, editors, *Proceedings of the 19th International Workshop on Satisfiability*
650 *Modulo Theories*, volume 2908 of *CEUR Workshop Proceedings*, pages 32–39, Online (initially
651 located in Los Angeles, USA), July 2021. CEUR. ISSN: 1613-0073. URL: [https://ceur-ws.](https://ceur-ws.org/Vol-2908/#short4)
652 [org/Vol-2908/#short4](https://ceur-ws.org/Vol-2908/#short4).
- 653 23 Jürgen Christ and Jochen Hoenicke. Weakly equivalent arrays. In Carsten Lutz and Silvio
654 Ranise, editors, *Frontiers of Combining Systems*, pages 119–134, Cham, 2015. Springer
655 International Publishing. doi:10.1007/978-3-319-24246-0_8.
- 656 24 Hélène Collavizza, Michel Rueher, and Pascal Van Hentenryck. CPBPV: A constraint-
657 programming framework for bounded program verification. *Constraints*, 15(2):238–264, April
658 2010. doi:10.1007/s10601-009-9089-9.
- 659 25 Sylvain Conchon, Evelyne Contejean, Johannes Kanig, and Stéphane Lescuyer. Cc (x):
660 Semantic combination of congruence closure with solvable theories. *Electronic Notes in*
661 *Theoretical Computer Science*, 198(2):51–69, 2008.
- 662 26 Leonardo De Moura and Nikolaj S Bjørner. Efficient e-matching for smt solvers. In *CADE*,
663 volume 4603, pages 183–198. Springer, 2007.
- 664 27 Leonardo de Moura and Dejan Jovanović. A model-constructing satisfiability calculus. In
665 Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni, editors, *Verification, Model Check-*
666 *ing, and Abstract Interpretation*, pages 1–12, Berlin, Heidelberg, 2013. Springer Berlin Heidel-
667 berg. doi:10.1007/978-3-642-35873-9_1.
- 668 28 Leonardo Mendonça de Moura and Nikolaj S. Bjørner. Generalized, efficient array decision
669 procedures. In *Proceedings of 9th International Conference on Formal Methods in Computer-*

- 670 *Aided Design, FMCAD 2009, 15-18 November 2009, Austin, Texas, USA*, pages 45–52. IEEE,
671 2009. doi:10.1109/FMCAD.2009.5351142.
- 672 29 Jean-Christophe Filliâtre and Claude Marché. The why/krakatoa/caduceus platform for
673 deductive program verification: (tool paper). In *International Conference on Computer Aided*
674 *Verification*, pages 173–177. Springer, 2007.
- 675 30 Harald Ganzinger, George Hagen, Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli.
676 DPLL(T): Fast Decision Procedures. In Rajeev Alur and Doron A. Peled, editors, *Computer*
677 *Aided Verification*, number 3114 in Lecture Notes in Computer Science, pages 175–188. Springer
678 Berlin Heidelberg, January 2004.
- 679 31 Arnaud Gotlieb. Euclide: A constraint-based testing framework for critical c programs. In
680 *2009 International Conference on Software Testing Verification and Validation*, pages 151–160.
681 IEEE, 2009.
- 682 32 Philippe Granger. Static analysis of arithmetical congruences. *International Journal of*
683 *Computer Mathematics*, 30(3-4):165–190, January 1989. doi:10.1080/00207168908803778.
- 684 33 Joxan Jaffar, Michael Maher, Kim Marriott, and Peter Stuckey. The semantics of constraint
685 logic programs. *The Journal of Logic Programming*, 37(1):1–46, October 1998. doi:10.1016/
686 S0743-1066(98)10002-X.
- 687 34 Fredrik Johansson. Calcium: computing in exact real and complex fields. In *Proceedings of*
688 *the 2021 on International Symposium on Symbolic and Algebraic Computation*, pages 225–232,
689 2021.
- 690 35 Dejan Jovanović and Leonardo de Moura. Solving non-linear arithmetic. In Bernhard Gramlich,
691 Dale Miller, and Uli Sattler, editors, *Automated Reasoning*, pages 339–354, Berlin, Heidelberg,
692 2012. Springer Berlin Heidelberg. doi:10.1007/978-3-642-31365-3_27.
- 693 36 Dejan Jovanovic and Bruno Dutertre. Libpoly: A library for reasoning about polynomials. In
694 *SMT*, pages 28–39, 2017.
- 695 37 Michel Leconte and Bruno Berstel. Domaines de congruence pour la programmation par
696 contraintes. In *Troisièmes Journées Francophones de Programmation par Contraintes (JFPC07)*,
697 2007.
- 698 38 K. R. M. Leino and Clément Pit-Claudel. Trigger selection strategies to stabilize program
699 verifiers. In Swarat Chaudhuri and Azadeh Farzan, editors, *Computer Aided Verification*, pages
700 361–381, Cham, 2016. Springer International Publishing. doi:10.1007/978-3-319-41528-4_
701 20.
- 702 39 Xavier Leroy, Damien Doligez, Alain Frisch, Jacques Garrigue, Didier Rémy, and Jérôme
703 Vouillon. The ocaml system: Documentation and user’s manual. *INRIA*, 3:42.
- 704 40 Dorian Lesbre, Matthieu Lemerre, Hichem Rami Ait-El-Hara, and François Bobot. Rela-
705 tional Abstractions Based on Labeled Union-Find. *Proceedings of the ACM on Program-*
706 *ming Languages*, 9(PLDI), June 2025. 46th ACM SIGPLAN Conference on Programming
707 Language Design and Implementation (PLDI 2025), Seoul, South Korea, 16-20/06/2025.
708 doi:10.1145/3729298.
- 709 41 Bruno Marre, François Bobot, and Zakaria Chihani. Real behavior of floating point numbers.
710 In *The SMT Workshop*, 2017.
- 711 42 Antoine Miné. Tutorial on static inference of numeric invariants by abstract inter-
712 pretation. *Foundations and Trends in Programming Languages*, 4(3-4):120–372, Decem-
713 ber 2017. arXiv:https://www.emerald.com/ftpgl/article-pdf/4/3-4/120/11044131/
714 2500000034en.pdf, doi:10.1561/25000000034.
- 715 43 Charles Gregory Nelson. *Techniques for program verification*. Stanford University, 1980.
- 716 44 Greg Nelson and Derek C. Oppen. Simplification by cooperating decision procedures. *ACM*
717 *Trans. Program. Lang. Syst.*, 1(2):245–257, October 1979. doi:10.1145/357073.357079.
- 718 45 Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and
719 Guido Tack. MiniZinc: Towards a Standard CP Modelling Language. In Christian Bessière,

- editor, *Principles and Practice of Constraint Programming – CP 2007*, volume 4741, pages 529–543. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. doi:10.1007/978-3-540-74970-7_38.
- 46 Aina Niemetz and Mathias Preiner. Bitwuzla. In Constantin Enea and Akash Lal, editors, *Computer Aided Verification*, pages 3–17, Cham, 2023. Springer Nature Switzerland.
- 47 Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T). *J. ACM*, 53(6):937–977, November 2006. doi:10.1145/1217856.1217859.
- 48 Jean-Charles Régin and Arnaud Malapert. Parallel Constraint Programming. In Youssef Hamadi and Lakhdar Sais, editors, *Handbook of Parallel Constraint Reasoning*, pages 337–379. Springer International Publishing, Cham, 2018. doi:10.1007/978-3-319-63516-3_9.
- 49 Francesca Rossi, Peter Van Beek, and Toby Walsh. *Handbook of Constraint Programming*, volume 2 of *Foundations of Artificial Intelligence*. Elsevier, 2006. doi:10.1016/S1574-6526(06)X8001-X.
- 50 Jean-Charles Régin. Global constraints and filtering algorithms. *Operations Research/Computer Science Interfaces Series*, 01 2003. doi:10.1007/978-1-4419-8917-8_4.
- 51 J. Schimpf and K. Shen. ECLiPSe - from LP to CLP. *Theory and Practice of Logic Programming*, 12(1-2):127–156, 2011.
- 52 Natarajan Shankar and Harald Rueß. Combining shostak theories. In *Rewriting Techniques and Applications: 13th International Conference, RTA 2002 Copenhagen, Denmark, July 22–24, 2002 Proceedings*, pages 1–18. Springer, 2002.
- 53 Robert E Shostak. Deciding combinations of theories. *Journal of the ACM (JACM)*, 31(1):1–12, 1984. doi:10.1145/2422.322411.
- 54 Peter J. Stuckey and Guido Tack. MiniZinc with Functions. In Carla Gomes and Meinolf Sellmann, editors, *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, volume 7874, pages 268–283. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013. doi:10.1007/978-3-642-38171-3_18.
- 55 George Varghese and Tony Lauck. Hashed and hierarchical timing wheels: Data structures for the efficient implementation of a timer facility. In *Proceedings of the eleventh ACM Symposium on Operating systems principles*, pages 25–38, 1987.

A Formalization details

A.1 Architecture

A.1.1 State

■ $\text{reprs} : \text{Node} \rightarrow \text{Node}$

A mapping representing the equivalence classes of nodes, it associates each node n to the representative of its equivalence class r , which is by default n itself, i.e.:

$$\text{reprs}[n] = \begin{cases} r & \text{if } n \in \text{Dom}(\text{reprs}) \text{ and } \text{reprs}[n] = r \\ n & \text{otherwise} \end{cases}$$

■ $\text{doms} : \text{Node} \rightarrow \mathcal{D} \rightarrow \mathcal{R}_{\mathcal{D}}$

A partial mapping of each representative node r to a mapping of each domain identifier $\mathcal{D}$ to the domain’s representation for the node r if it exists.

■ $\text{contradiction} : \text{Bool}$

A flag indicates if a contradiction has been found (the implementation raises an exception in this case)

A.1.2 Domains

► **Definition 3** (Domains in colibri2). A domain $\mathcal{D}$ in colibri2 can be formalized as a tuple $\langle \mathcal{R}, P_{\mathcal{A}}, \text{equal} \rangle$ where:

■ $\mathcal{R}_{\mathcal{D}}$: the set of domain representants.

■ $P_{\mathcal{D}}^{\mathcal{A}} : \mathcal{R} \rightarrow 2^A$

Given a model $\mathcal{A}$ and an element of the domain $d \in \mathcal{R}$, $P_{\mathcal{A}}(d)$ denotes the set of values in the universe A represented by d under $\mathcal{A}$.

■ $\text{equal}_{\mathcal{D}} : \mathcal{R} \rightarrow \mathcal{R} \rightarrow \text{Bool}$

Checks whether two domain representations have the same semantic interpretation for all domains. It is defined for any $d_1, d_2 \in \mathcal{R}$ as:

$$\text{equal}(d_1, d_2) \implies \forall \mathcal{A}. P_{\mathcal{A}}(d_1) = P_{\mathcal{A}}(d_2)$$

For convenience, it will be denoted as an equality “=” between domain representations.

In the formalization of domains in colibri2 presented in Definition 3, $P_{\mathcal{A}}$ depends on the model $\mathcal{A}$ in order to support relational domains, which are domains whose elements can contain other nodes. An example of such a domain is the linear form domain, where the linear form representation r of a node can contain other nodes, therefore, to compute $P_{\mathcal{A}}(r)$, a model $\mathcal{A}$ is needed to retrieve the values of the nodes that may appear in r .

The interpretation of a model $\mathcal{A}$ is extended to environment env .

$$(\text{env})^{\mathcal{A}} = \top \triangleq \begin{array}{l} \forall n \in \text{Node}. (\text{env.reprs}[n])^{\mathcal{A}} = (n)^{\mathcal{A}} \\ \forall \mathcal{D}, \forall n \in \text{Node}. (n)^{\mathcal{A}} \in P_{\mathcal{D}}^{\mathcal{A}}(\text{env.doms}(n, \mathcal{D})^{\mathcal{A}}) \end{array}$$

► **Definition 4** (The domain definition $\mathcal{D}_{\text{merge}}$). A domain $\mathcal{D}$ in colibri2 can be defined as the tuple $\langle \mathcal{R}, P_{\mathcal{A}}, \text{equal}, \text{merge}_{\mathcal{D}} \rangle$, where $\mathcal{R}$, $P_{\mathcal{A}}$, and equal are the same as in Definition 1, and merge is:

■ $\text{merge}_{\mathcal{D}} : \text{env} \rightarrow (n_1 : \text{Node}) \rightarrow (n_2 : \text{Node}) \rightarrow \text{env}$

Ensures that the nodes n_1 and n_2 have the same domain $\mathcal{D}$ in env . The function must follow the following soundness property:

$$\forall \mathcal{A}, \text{env}, n_1, n_2. (\text{env})^{\mathcal{A}} = \top \implies (n_1)^{\mathcal{A}} = (n_2)^{\mathcal{A}} \implies (\text{merge}_{\mathcal{D}}(\text{env}, n_1, n_2))^{\mathcal{A}}$$

and must ensure in the resulting state env' that $\text{env}'.\text{dom}(n_1, \mathcal{D}) = \text{env}'.\text{dom}(n_2, \mathcal{D})$.

When a domain is defined using $\mathcal{D}_{\text{merge}}$, it can be interacted with in the implementation through two functions:

■ $\text{get_dom}_{\mathcal{D}} : \text{env} \rightarrow (n : \text{Node}) \rightarrow \mathcal{R}_{\mathcal{D}} \text{ option}$

Returns $\text{env.doms}[n][\mathcal{D}]$, i.e. the domain $\mathcal{D}$ of the node n if it is set.

■ $\text{set_dom}_{\mathcal{D}} : \text{env} \rightarrow (n : \text{Node}) \rightarrow (d : \mathcal{R}_{\mathcal{D}}) \rightarrow \text{env}$

Sets the domain $\mathcal{D}$ of the node n with the domain representation d . Ensures that: $\text{env.doms}[n][\mathcal{D}] = d$

► **Remark 5.** A domain $\mathcal{D}$ can be defined using a classical intersection function $\text{inter}_{\mathcal{D}} : \mathcal{R}_{\mathcal{D}} \rightarrow \mathcal{R}_{\mathcal{D}} \rightarrow \mathcal{R}_{\mathcal{D}}^{\perp}$:

$$\text{merge}_{\mathcal{D}}(\text{env}, n_1, n_2) = \begin{cases} \text{env} & \text{if } (n_1, \mathcal{D}) \notin \text{env.doms} \text{ and } (n_2, \mathcal{D}) \notin \text{env.doms} \\ \text{env}[\text{doms}(n_1, \mathcal{D}) \leftarrow (n_2, \mathcal{D})] & \text{if } (n_2, \mathcal{D}) \notin \text{env.doms} \\ \text{env}[\text{doms}(n_2, \mathcal{D}) \leftarrow (n_1, \mathcal{D})] & \text{if } (n_1, \mathcal{D}) \notin \text{env.doms} \\ \text{env} = \text{env}[\text{contradiction} \leftarrow \text{true}] & \text{if } \perp = \text{inter}_{\mathcal{D}}(\text{env.doms}(n_1, \mathcal{D}), \text{env.doms}(n_2, \mathcal{D})) \\ \text{env}[r \leftarrow (n_1, \mathcal{D}), r \leftarrow (n_2, \mathcal{D})] & \text{if } r = \text{inter}_{\mathcal{D}}(\text{env.doms}(n_1, \mathcal{D}), \text{env.doms}(n_2, \mathcal{D})) \end{cases}$$

791 The soundness of $\text{merge}_{\mathcal{D}}$ is direct if $\text{inter}_{\mathcal{D}}$ verifies the following soundness property
 792 $\forall r_1, r_2 \in \mathcal{R}_{\mathcal{D}}, P_{\mathcal{D}}^A(r_1) \cap P_{\mathcal{D}}^A(r_2) \subset P_{\mathcal{D}}^A(\text{inter}_{\mathcal{D}}(r_1, r_2))$

793 The domains' merge functions are used when merging two nodes. When the merger of
 794 two nodes is requested, before updating their representatives in `env.reprs`, their domains are
 795 first conciliated using the domains' respective merge functions, ensuring that they have the
 796 same domain representations for all domains before merging. An error is raised otherwise.

797 A.2 Disequalities

798 More formally $R_{\text{Dis}} = \text{Node} \rightarrow \text{Node}$ is the set of partial maps from node to node, $\text{equal}_{\text{Dis}}$ is
 799 the equality on partial maps,

$$\begin{aligned}
 800 \quad P_{\text{Dis}}^A(m) &= \{v \mid \forall(_, n) \in m. v = (n)^A\} \\
 801 \quad \text{merge}_{\text{Dis}}(\text{env}, n_1, n_2) &= \text{set_dom}(\text{merge}(\text{env}, s), m_3) \quad \text{with} \\
 802 \quad s &= \{(k, \top) \mid \forall n_3, n_4. (k, n_3) \in m_1 \wedge (k, n_4) \in m_2 \wedge n_3 \neq n_4\} \\
 803 \quad m_3 &= \{(k, n) \mid ((k, n) \in m_1 \wedge (k, _) \notin m_2) \vee (k, n) \in m_2\}
 \end{aligned}$$